

Miniproject evidence

team miniproject20: wenwenw, junwooh, bolanz, pbu
generated 2026-10-09 20:13 EDT on macOS-15.7.7-arm64-arm-64bit, polars 2.0.0, numpy 2.5.3
project 06763-miniproject
evidence script sha256 744445fa94498271372f89d7a8053499fd20999c7754842f770eb7bd5f4a157a

Score **10.0 / 10** automatic, of 15 for the miniproject.
REPORT.pdf is worth 5, is submitted separately, and is read by your TA.

What this script found

free	data/tep_fault_free_training.parquet
faulty	data/tep_faulty_training_runs01-20.parquet
pca	results/scores_pca.parquet
ridge	results/scores_ridge.parquet
thresholds	results/thresholds.csv
detection	results/detection.csv
contributions	results/contributions.csv

Detectors 5.0 / 5

[PASS] scores_pca.parquet present
[PASS] every required PCA row scored
100.00% covered, 0 extra rows
[PASS] T2 matches the reference
100.00% of rows within 0.1 %
[PASS] SPE matches the reference
100.00% of rows within 0.1 %
[PASS] scores_ridge.parquet present
[PASS] every required ridge row scored
100.00% covered, 0 extra rows
[PASS] ridge score matches the reference
100.00% of rows within 0.1 %
[PASS] a ridge regression fit in the code

Evaluation and diagnosis 5.0 / 5

[PASS] thresholds.csv with T2, SPE and ridge
[PASS] thresholds are the 99th percentile of the team's validation scores
agree for ['T2', 'SPE', 'ridge']
[PASS] detection.csv matches a recomputation from the team's scores
100% of fault/detector detection rates agree within 0.001
[PASS] false-alarm rows (fault 0) reported for each detector
[PASS] detection.csv covers faults 1 to 20 for all three detectors
[PASS] faults 3, 9 and 15 reported as not detected
largest rate 0.000
[PASS] contributions.csv present
[PASS] top channels match the reference
100% of fault/detector top channels agree

REPORT.pdf 5 points, read by your TA

[SKIP] REPORT.pdf, submitted separately and read by your TA

Transcript

data

tep_fault_free_training.parquet: tep_fault_free_training.parquet, sha256 matches the published file
tep_faulty_training_runs01-20.parquet: tep_faulty_training_runs01-20.parquet, sha256 matches the published file

the script's own detectors

PCA: 31 components reach 90% of the variance
ridge: 2 lags of 52 channels, alpha 1.0

thresholds (99th percentile of validation runs): T2 51.7971, SPE 11.7745, ridge 112.4423
rows to score: 300,000 (validation, test, faults 1-20 runs 1-20)

scores_pca.parquet against the reference

```
{
  "rows_mine": 300000,
  "rows_ref": 300000,
  "covered": 1.0,
  "extra": 0,
  "t2": 1.0,
  "spe": 1.0
}
```

scores_ridge.parquet against the reference

```
{
  "rows_mine": 298800,
  "rows_ref": 298800,
  "covered": 1.0,
  "extra": 0,
  "score": 1.0
}
```

detection.csv against a recomputation

```
100% of fault/detector detection rates agree within 0.001
100% of median delays agree
shape: (12, 7)
????????????????????????????????????????????????????????????????????????????????????
? fault ? detector ? detection_rate ? median_delay_min ? runs_missed ? rate_mine ? delay_mine ?
? --- ? --- ? --- ? --- ? --- ? --- ? --- ?
? i64 ? str ? f64 ? f64 ? i64 ? f64 ? f64 ?
????????????????????????????????????????????????????????????????????????????????????
? 0 ? T2 ? 0.00052 ? null ? null ? 0.00052 ? null ?
? 1 ? T2 ? 0.985417 ? 25.5 ? 0 ? 0.985417 ? 25.5 ?
? 2 ? T2 ? 0.972813 ? 39.0 ? 0 ? 0.972813 ? 39.0 ?
? 3 ? T2 ? 0.000208 ? 231.0 ? 19 ? 0.000208 ? 231.0 ?
? 4 ? T2 ? 0.091354 ? 64.5 ? 0 ? 0.091354 ? 64.5 ?
? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ?
? 7 ? T2 ? 0.995937 ? 9.0 ? 0 ? 0.995937 ? 9.0 ?
? 8 ? T2 ? 0.941875 ? 70.5 ? 0 ? 0.941875 ? 70.5 ?
? 9 ? T2 ? 0.000104 ? 231.0 ? 19 ? 0.000104 ? 231.0 ?
? 10 ? T2 ? 0.084896 ? 471.0 ? 0 ? 0.084896 ? 471.0 ?
? 11 ? T2 ? 0.257708 ? 43.5 ? 0 ? 0.257708 ? 43.5 ?
????????????????????????????????????????????????????????????????????????????????????
```

contributions.csv against the reference

```
34 of 34 top channels agree
fault 1 SPE reference xmeas_4 yours xmeas_4
fault 1 ridge reference xmeas_1 yours xmeas_1
fault 2 SPE reference xmeas_25 yours xmeas_25
fault 2 ridge reference xmeas_10 yours xmeas_10
fault 4 SPE reference xmv_10 yours xmv_10
fault 4 ridge reference xmv_10 yours xmv_10
fault 5 SPE reference xmeas_38 yours xmeas_38
fault 5 ridge reference xmeas_1 yours xmeas_1
fault 6 SPE reference xmv_5 yours xmv_5
fault 6 ridge reference xmeas_1 yours xmeas_1
fault 7 SPE reference xmeas_4 yours xmeas_4
fault 7 ridge reference xmv_4 yours xmv_4
```

the script's detection table, for comparison

```
shape: (21, 4)
????????????????????????????????????????????
```

```

? fault ? T2      ? SPE      ? ridge  ?
? ---   ? ---     ? ---     ? ---   ?
? i64   ? f64     ? f64     ? f64   ?
????????????????????????????????????????
? 0      ? 0.00052 ? 0.0      ? 0.0    ?
? 1      ? 0.985417 ? 0.991875 ? 0.993542 ?
? 2      ? 0.972813 ? 0.958958 ? 0.965104 ?
? 3      ? 0.000208 ? 0.0      ? 0.0    ?
? 4      ? 0.091354 ? 0.994167 ? 0.968854 ?
? ?      ? ?       ? ?       ? ?       ?
? 16     ? 0.019375 ? 0.101146 ? 0.711875 ?
? 17     ? 0.665938 ? 0.8678125 ? 0.866875 ?
? 18     ? 0.870833 ? 0.884375 ? 0.892708 ?
? 19     ? 0.009896 ? 0.003333 ? 0.029167 ?
? 20     ? 0.13625  ? 0.432188 ? 0.635104 ?
????????????????????????????????????????

```

Code: common.py

```

"""Paths, data splits and the standardization shared by the PCA and forecast monitors."""

from __future__ import annotations

from pathlib import Path

import numpy as np
import polars as pl

ROOT = Path(__file__).resolve().parent
DATA = ROOT / "data"
FREE = DATA / "tep_fault_free_training.parquet"
FAULTY = DATA / "tep_faulty_training_runs01-20.parquet"
RESULTS = ROOT / "results"
FIGURES = ROOT / "figures"

FAULT = "faultNumber"
RUN = "simulationRun"
SAMPLE = "sample"
KEYS = [FAULT, RUN, SAMPLE]
CHANNELS = [f"xmeas_{i}" for i in range(1, 42)] + [f"xmv_{i}" for i in range(1, 12)]

TRAIN = (1, 300)
VALIDATION = (301, 400)
TEST = (401, 500)
FAULTS = (1, 20)
FAULT_RUNS = (1, 20)
FAULT_START = 20
MINUTES_PER_SAMPLE = 3

def load() -> tuple[pl.DataFrame, pl.DataFrame]:
    """Return training runs 1 to 300 and the validation, test and faulty rows, both sorted by fault, run
    and sample."""
    free = pl.read_parquet(FREE, columns=KEYS + CHANNELS)
    faulty = pl.read_parquet(FAULTY, columns=KEYS + CHANNELS)
    train = free.filter(pl.col(RUN).is_between(*TRAIN)).sort(KEYS)
    scored = pl.concat([
        free.filter(pl.col(RUN).is_between(VALIDATION[0], TEST[1])),
        faulty.filter(pl.col(FAULT).is_between(*FAULTS) & pl.col(RUN).is_between(*FAULT_RUNS)),
    ]).sort(KEYS)
    return train, scored

class Standardizer:
    """Per-channel mean and standard deviation (ddof=1) from the training runs."""

    def __init__(self, train: pl.DataFrame):

```

```

x = train.select(CHANNELS).to_numpy()
self.mean = x.mean(axis=0)
self.std = x.std(axis=0, ddof=1)

def transform(self, frame: pl.DataFrame) -> pl.DataFrame:
    """Return the keys and the 52 standardized channels."""
    z = (frame.select(CHANNELS).to_numpy() - self.mean) / self.std
    return frame.select(KEYS).with_columns(
        [pl.Series(c, z[:, j]) for j, c in enumerate(CHANNELS)])

def split_of(frame: pl.DataFrame) -> pl.DataFrame:
    """Add a split column: validation, test or faulty."""
    return frame.with_columns(
        split=pl.when(pl.col(FAULT) > 0).then(pl.lit("faulty"))
        .when(pl.col(RUN).is_between(*VALIDATION)).then(pl.lit("validation"))
        .otherwise(pl.lit("test")))

```

Code: diagnosis.py

```

"""Diagnosis: the channels that drive the SPE and ridge alarms, per fault.

```

SPE and the ridge score are sums of squares over the 52 channels, so each splits into one contribution per channel. For each fault and detector, the contributions are averaged over every sample in alarm after sample 20, pooled over the 20 runs, and the five largest are kept.

The alarms are the evaluation's: evaluate.py's threshold and three-in-a-row rule, applied to the scores in results/, so the samples averaged here are the ones counted in detection.csv.

```

from __future__ import annotations

```

```

import numpy as np
import polars as pl

```

```

from common import CHANNELS, FAULT, FAULT_START, FAULTS, KEYS, RESULTS, RUN, SAMPLE, Standardizer, load
from evaluate import alarm_flags, validation_threshold
from forecast_monitor import ForecastMonitor, make_table
from pca_monitor import fit_pca, spe_contributions

```

```

TOP = 5

```

```

SCORES = {"SPE": ("scores_pca.parquet", "SPE"), "ridge": ("scores_ridge.parquet", "score")}

```

```

def channel_terms(train: pl.DataFrame, faulty: pl.DataFrame) -> dict[str, tuple[pl.DataFrame, np.ndarray]]:
    """Return, for SPE and ridge, the keys of the faulty rows and each channel's term of the statistic.

```

```

    SPE:  $(z_j - (P^T z)_j)^2$  from the PCA monitor. Ridge: the squared standardized residual of
    channel j from the forecast monitor. Each row of terms sums to that row's SPE or ridge score.
    """

```

```

    std = Standardizer(train)
    z_train, z = std.transform(train), std.transform(faulty)
    table = make_table(z)
    return {
        "SPE": (z.select(KEYS), spe_contributions(z, fit_pca(z_train))),
        "ridge": (table.select(KEYS), ForecastMonitor().fit(make_table(z_train)).contributions(table)),
    }

```

```

def alarm_samples(detector: str, keys: pl.DataFrame, terms: np.ndarray) -> np.ndarray:
    """Return which faulty rows are in alarm after sample 20, by the evaluation's rule on the scores in
    results/.

```

```

    Raises if the channel terms do not add up to those scores, i.e. if results/ is out of date.
    """

```

```

    name, column = SCORES[detector]
    scores = pl.read_parquet(RESULTS / name)

```

```

threshold = validation_threshold(scores, column)
faulty = scores.filter(pl.col(FAULT) > 0).sort(KEYS)
if not faulty.select(KEYS).equals(keys):
    raise ValueError(f"results/{name} does not hold the faulty rows the monitor scores")
score = faulty[column].to_numpy()
gap = float(np.max(np.abs(terms.sum(axis=1) - score) / score))
if gap > 1e-6:
    raise ValueError(f"the channel terms do not add up to results/{name} (relative gap {gap:.1e})")
print(f"{detector}: threshold {threshold!r}; terms add up to results/{name} to a relative {gap:.1e}")

flags = np.concatenate([alarm_flags(g[column].to_numpy(), threshold)
                        for _, g in faulty.group_by([FAULT, RUN], maintain_order=True)])
return flags & (faulty[SAMPLE].to_numpy() > FAULT_START)

def mean_contributions(detector: str, keys: pl.DataFrame, terms: np.ndarray) -> pl.DataFrame:
    """Average each channel's term over the alarm samples after sample 20 of each fault, pooled over its
    runs.

    Faults with no such sample are left out, since their average is undefined.
    """
    keep = alarm_samples(detector, keys, terms)
    fault = keys[FAULT].to_numpy()

    frames = []
    for f in range(FAULTS[0], FAULTS[1] + 1):
        m = keep & (fault == f)
        if m.any():
            frames.append(pl.DataFrame({"channel": CHANNELS, "contribution": terms[m].mean(axis=0)}).
                           .with_columns(fault=pl.lit(f), detector=pl.lit(detector),
                                          samples=pl.lit(int(m.sum()))))
    return pl.concat(frames).select("fault", "detector", "channel", "contribution", "samples")

def mean_table() -> pl.DataFrame:
    """Return the mean contribution of every channel, per fault and detector."""
    train, scored = load()
    terms = channel_terms(train, scored.filter(pl.col(FAULT) > 0))
    return pl.concat([mean_contributions(detector, keys, t) for detector, (keys, t) in terms.items()])

def top_channels(means: pl.DataFrame) -> pl.DataFrame:
    """Return the TOP channels with the largest mean contribution per fault and detector, ranked from 1."""
    return (means.sort(["fault", "detector", "contribution"], descending=[False, False, True])
            .group_by(["fault", "detector"], maintain_order=True).head(TOP)
            .with_columns(rank=pl.int_range(1, pl.len() + 1).over("fault", "detector"))
            .select("fault", "detector", "rank", "channel", "contribution"))

def run() -> pl.DataFrame:
    """Write results/contributions.csv; print the thresholds and the leading channel per fault and
    detector."""
    means = mean_table()
    top = top_channels(means)
    RESULTS.mkdir(exist_ok=True)
    top.write_csv(RESULTS / "contributions.csv")

    lead = (top.filter(pl.col("rank") == 1)
            .join(means.select("fault", "detector", "samples").unique(), on=["fault", "detector"])
            .sort("fault", "detector"))
    with pl.Config(tbl_rows=-1):
        print(lead.select("fault", "detector", "samples", "channel", "contribution"))
    have = set(zip(lead["fault"].to_list(), lead["detector"].to_list()))
    empty = [f"{f}/{d}" for f in range(FAULTS[0], FAULTS[1] + 1) for d in ("SPE", "ridge") if (f, d) not in
             have]
    print(f"{top.height} rows written; no alarm sample after sample {FAULT_START}: {'', '.join(empty) or
    'none'}")

```

```
    return top
```

```
if __name__ == "__main__":  
    run()
```

Code: evaluate.py

```
"""Evaluate the PCA and ridge fault monitors."""  
  
from __future__ import annotations  
  
import numpy as np  
import polars as pl  
  
from common import KEYS, RESULTS  
  
VALIDATION_RUNS = (301, 400)  
TEST_RUNS = (401, 500)  
  
FAULT_ONSET_SAMPLE = 20  
MINUTES_PER_SAMPLE = 3  
CONSECUTIVE_EXCEEDANCES = 3  
  
DETECTORS = ("T2", "SPE", "ridge")  
FAULTS = range(1, 21)  
  
def alarm_flags(  
    scores: np.ndarray,  
    threshold: float,  
    ) -> np.ndarray:  
    """Return alarms produced by three consecutive threshold exceedances."""  
  
    exceedances = scores > threshold  
  
    consecutive = np.convolve(  
        exceedances.astype(int),  
        np.ones(CONSECUTIVE_EXCEEDANCES, dtype=int),  
        mode="valid",  
    )  
  
    alarms = consecutive >= CONSECUTIVE_EXCEEDANCES  
  
    return np.concatenate(  
        [  
            np.zeros(CONSECUTIVE_EXCEEDANCES - 1, dtype=bool),  
            alarms,  
        ]  
    )  
  
def validation_threshold(  
    scores: pl.DataFrame,  
    score_column: str,  
    ) -> float:  
    """Calculate the validation-set 99th percentile."""  
  
    validation_scores = (  
        scores.filter(  
            (pl.col("faultNumber") == 0)  
            & pl.col("simulationRun").is_between(  
                *VALIDATION_RUNS  
            )  
        )  
        .get_column(score_column)
```

```

        .to_numpy()
    )

    if validation_scores.size == 0:
        raise ValueError(
            f"No validation scores found for {score_column}."
        )

    return float(np.quantile(validation_scores, 0.99))

def detector_metrics(
    scores: pl.DataFrame,
    score_column: str,
    detector: str,
    threshold: float,
) -> list[dict[str, object]]:
    """Calculate false alarms, detection rates and detection delays."""

    per_run: list[tuple[int, float, float | None]] = []

    grouped = (
        scores.select(KEYS + [score_column])
        .sort(KEYS)
        .group_by(
            ["faultNumber", "simulationRun"],
            maintain_order=True,
        )
    )

    for (fault, run), run_scores in grouped:
        values = run_scores.get_column(score_column).to_numpy()
        samples = run_scores.get_column("sample").to_numpy()

        alarms = alarm_flags(values, threshold)

        if fault == 0:
            if TEST_RUNS[0] <= run <= TEST_RUNS[1]:
                per_run.append(
                    (
                        0,
                        float(alarms.mean()),
                        None,
                    )
                )

            continue

        post_fault = samples > FAULT_ONSET_SAMPLE
        post_fault_alarms = alarms & post_fault

        detection_rate = float(
            alarms[post_fault].mean()
        )

        alarm_positions = np.flatnonzero(
            post_fault_alarms
        )

        if alarm_positions.size:
            first_alarm_sample = samples[
                alarm_positions[0]
            ]

            delay = float(
                (
                    first_alarm_sample

```

```

        - FAULT_ONSET_SAMPLE
    )
    * MINUTES_PER_SAMPLE
)
else:
    delay = None

per_run.append(
    (
        int(fault),
        detection_rate,
        delay,
    )
)

rows: list[dict[str, object]] = []

false_alarm_runs = [
    row for row in per_run if row[0] == 0
]

if len(false_alarm_runs) != 100:
    raise ValueError(
        f"{detector}: expected 100 test runs, "
        f"found {len(false_alarm_runs)}."
    )

rows.append(
    {
        "fault": 0,
        "detector": detector,
        "detection_rate": float(
            np.mean(
                [
                    row[1]
                    for row in false_alarm_runs
                ]
            )
        ),
        "median_delay_min": None,
        "runs_missed": None,
    }
)

for fault in FAULTS:
    fault_runs = [
        row for row in per_run
        if row[0] == fault
    ]

    if len(fault_runs) != 20:
        raise ValueError(
            f"{detector}, fault {fault}: "
            f"expected 20 runs, "
            f"found {len(fault_runs)}."
        )

    detected_delays = [
        row[2]
        for row in fault_runs
        if row[2] is not None
    ]

    rows.append(
        {
            "fault": fault,
            "detector": detector,

```

```

        "detection_rate": float(
            np.mean(
                [
                    row[1]
                    for row in fault_runs
                ]
            )
        ),
        "median_delay_min": (
            float(
                np.median(
                    detected_delays
                )
            )
            if detected_delays
            else None
        ),
        "runs_missed": sum(
            row[2] is None
            for row in fault_runs
        ),
    }
}

return rows

```

```

def run() -> None:
    """Generate thresholds.csv and detection.csv."""

    pca_path = RESULTS / "scores_pca.parquet"
    ridge_path = RESULTS / "scores_ridge.parquet"

    pca_scores = pl.read_parquet(pca_path)
    ridge_scores = pl.read_parquet(ridge_path)

    thresholds = {
        "T2": validation_threshold(
            pca_scores,
            "T2",
        ),
        "SPE": validation_threshold(
            pca_scores,
            "SPE",
        ),
        "ridge": validation_threshold(
            ridge_scores,
            "score",
        ),
    }

    threshold_table = pl.DataFrame(
        {
            "detector": list(thresholds),
            "threshold": list(thresholds.values()),
        }
    )

    metric_rows: list[dict[str, object]] = []

    metric_rows.extend(
        detector_metrics(
            pca_scores,
            "T2",
            "T2",
            thresholds["T2"],
        )
    )

```

```

)

metric_rows.extend(
    detector_metrics(
        pca_scores,
        "SPE",
        "SPE",
        thresholds["SPE"],
    )
)

metric_rows.extend(
    detector_metrics(
        ridge_scores,
        "score",
        "ridge",
        thresholds["ridge"],
    )
)

detection_table = pl.DataFrame(
    metric_rows,
    schema={
        "fault": pl.Int64,
        "detector": pl.String,
        "detection_rate": pl.Float64,
        "median_delay_min": pl.Float64,
        "runs_missed": pl.Int64,
    },
).sort(["fault", "detector"])

if detection_table.height != 63:
    raise ValueError(
        "Expected 63 detection rows, "
        f"found {detection_table.height}."
    )

duplicate_rows = (
    detection_table.height
    - detection_table.unique(
        subset=["fault", "detector"]
    ).height
)

if duplicate_rows:
    raise ValueError(
        "Duplicate fault-detector rows found."
    )

RESULTS.mkdir(exist_ok=True)

threshold_path = RESULTS / "thresholds.csv"
detection_path = RESULTS / "detection.csv"

threshold_table.write_csv(threshold_path)
detection_table.write_csv(detection_path)

print("THRESHOLDS")
print(threshold_table)

print("\nFALSE-ALARM RATES")
print(
    detection_table.filter(
        pl.col("fault") == 0
    )
)

```

```

print("\nEXPECTED DIFFICULT FAULTS")
print(
    detection_table.filter(
        pl.col("fault").is_in([3, 9, 15])
    )
)

print(
    f"\nSaved thresholds to {threshold_path}"
)
print(
    f"Saved detection results to {detection_path}"
)

if __name__ == "__main__":
    run()

```

Code: forecast_monitor.py

```

"""Forecast-residual monitor: one ridge predicts all 52 standardized channels at t from t-1 and t-2."""

from __future__ import annotations

import numpy as np
import polars as pl
from sklearn.linear_model import Ridge

from common import CHANNELS, FAULT, KEYS, RESULTS, RUN, Standardizer, load

LAGS = 2
ALPHA = 1.0

def lag_names(lag: int) -> list[str]:
    """Return the column names of the 52 channels lagged by lag samples."""
    return [f"{c}_lag{lag}" for c in CHANNELS]

FEATURES = [c for lag in range(1, LAGS + 1) for c in lag_names(lag)]

def make_table(z: pl.DataFrame) -> pl.DataFrame:
    """Return the keys, the 104 lagged features and the 52 targets; the first LAGS samples of each run are dropped.

    z holds the keys and standardized channels, sorted by fault, run and sample.
    """
    lagged = [pl.col(c).shift(lag).over(FAULT, RUN).alias(name)
               for lag in range(1, LAGS + 1) for c, name in zip(CHANNELS, lag_names(lag))]
    return z.with_columns(lagged).drop_nulls(FEATURES)

class ForecastMonitor:
    """Ridge forecaster fitted on the training runs, with each channel's residual standard deviation."""

    def __init__(self, alpha: float = ALPHA):
        self.model = Ridge(alpha=alpha)
        self.res_std = None

    def fit(self, train: pl.DataFrame) -> ForecastMonitor:
        """Fit the ridge on the training table and store the residual standard deviation per channel."""
        x = train.select(FEATURES).to_numpy()
        y = train.select(CHANNELS).to_numpy()
        self.model.fit(x, y)
        self.res_std = (y - self.model.predict(x)).std(axis=0, ddof=1)
        return self

```

```

def contributions(self, table: pl.DataFrame) -> np.ndarray:
    """Return the squared standardized residual of every channel, one row per sample."""
    x = table.select(FEATURES).to_numpy()
    y = table.select(CHANNELS).to_numpy()
    r = (y - self.model.predict(x)) / self.res_std
    return r ** 2

def score(self, table: pl.DataFrame) -> pl.DataFrame:
    """Return the keys and score, the sum of the 52 squared standardized residuals."""
    return table.select(KEYS).with_columns(
        score=pl.Series(self.contributions(table).sum(axis=1)))

def run() -> pl.DataFrame:
    """Fit on runs 1 to 300 and write results/scores_ridge.parquet."""
    train, scored = load()
    std = Standardizer(train)
    monitor = ForecastMonitor().fit(make_table(std.transform(train)))
    scores = monitor.score(make_table(std.transform(scored)))

    RESULTS.mkdir(exist_ok=True)
    scores.write_parquet(RESULTS / "scores_ridge.parquet")
    print(f"{scores.height:,} rows scored")
    return scores

if __name__ == "__main__":
    run()

```

Code: `pca_monitor.py`

```

"""PCA process monitor using Hotelling's T-squared and SPE statistics."""

from __future__ import annotations

from dataclasses import dataclass

import numpy as np
import polars as pl

from common import (
    CHANNELS,
    KEYS,
    RESULTS,
    Standardizer,
    load,
)

VARIANCE_TARGET = 0.90
EXPECTED_SCORE_ROWS = 300_000

@dataclass(frozen=True)
class PCAModel:
    """Parameters required to apply the fitted PCA monitor."""

    all_eigenvalues: np.ndarray
    retained_eigenvalues: np.ndarray
    loadings: np.ndarray
    n_components: int
    cumulative_explained_variance: float

    def fit_pca(
        self,
        standardized_train: pl.DataFrame,
        variance_target: float = VARIANCE_TARGET,
    ) -> PCAModel:
        """Fit the PCA model on the standardized training data.

        Parameters
        -----
        standardized_train: pl.DataFrame
            The standardized training data.
        variance_target: float
            The target variance explained by the PCA model.

        Returns
        -----
        PCAModel
            The fitted PCA model.
        """
        # Fit the PCA model
        pca = PCA(
            n_components=self.n_components,
            variance_target=variance_target,
        )
        pca.fit(standardized_train)

        # Calculate the eigenvalues and loadings
        all_eigenvalues = pca.eigenvalues_
        retained_eigenvalues = pca.eigenvalues_[
            : self.n_components
        ]
        loadings = pca.components_.T

        # Calculate the cumulative explained variance
        cumulative_explained_variance = sum(
            retained_eigenvalues
        ) / sum(all_eigenvalues)

        # Create the PCAModel object
        return PCAModel(
            all_eigenvalues=all_eigenvalues,
            retained_eigenvalues=retained_eigenvalues,
            loadings=loadings,
            n_components=self.n_components,
            cumulative_explained_variance=cumulative_explained_variance,
        )

```

```

) -> PCAModel:
    """Fit PCA to standardized fault-free training observations."""

    z_train = standardized_train.select(CHANNELS).to_numpy()

    # np.cov uses sample normalization (N - 1) by default.
    covariance = np.cov(z_train, rowvar=False)

    # eig returns eigenvalues in ascending order.
    eigenvalues, eigenvectors = np.linalg.eigh(covariance)

    order = np.argsort(eigenvalues)[::-1]
    eigenvalues = eigenvalues[order]
    eigenvectors = eigenvectors[:, order]

    explained_ratio = eigenvalues / eigenvalues.sum()
    cumulative_variance = np.cumsum(explained_ratio)

    n_components = int(
        np.searchsorted(
            cumulative_variance,
            variance_target,
            side="left",
        )
        + 1
    )

    retained_eigenvalues = eigenvalues[:n_components]
    loadings = eigenvectors[:, :n_components]

    if np.any(retained_eigenvalues <= 0):
        raise ValueError("All retained eigenvalues must be positive.")

    return PCAModel(
        all_eigenvalues=eigenvalues,
        retained_eigenvalues=retained_eigenvalues,
        loadings=loadings,
        n_components=n_components,
        cumulative_explained_variance=float(
            cumulative_variance[n_components - 1]
        ),
    )

def statistics(
    standardized_frame: pl.DataFrame,
    model: PCAModel,
) -> tuple[np.ndarray, np.ndarray]:
    """Compute T-squared and SPE for every standardized observation."""

    z = standardized_frame.select(CHANNELS).to_numpy()

    component_scores = z @ model.loadings

    t_squared = np.sum(
        component_scores**2 / model.retained_eigenvalues,
        axis=1,
    )

    reconstruction = component_scores @ model.loadings.T
    residual = z - reconstruction
    spe = np.sum(residual**2, axis=1)

    return t_squared, spe

def spe_contributions(

```

```

        standardized_frame: pl.DataFrame,
        model: PCAModel,
    ) -> np.ndarray:
        """Return the per-channel squared PCA residual contributions."""

        z = standardized_frame.select(CHANNELS).to_numpy()
        component_scores = z @ model.loadings
        residual = z - component_scores @ model.loadings.T

        return residual**2

def score(
    standardized_frame: pl.DataFrame,
    model: PCAModel,
) -> pl.DataFrame:
    """Return identifiers, T-squared and SPE for every row."""

    t_squared, spe = statistics(
        standardized_frame,
        model,
    )

    return standardized_frame.select(KEYS).with_columns(
        pl.Series("T2", t_squared),
        pl.Series("SPE", spe),
    )

def validate_scores(scores: pl.DataFrame) -> None:
    """Check the required output structure before writing it."""

    expected_columns = [
        "faultNumber",
        "simulationRun",
        "sample",
        "T2",
        "SPE",
    ]

    if scores.columns != expected_columns:
        raise ValueError(
            f"Unexpected output columns: {scores.columns}"
        )

    if scores.height != EXPECTED_SCORE_ROWS:
        raise ValueError(
            f"Expected {EXPECTED_SCORE_ROWS:,} rows, "
            f"found {scores.height:,}."
        )

    unique_rows = scores.unique(subset=KEYS).height

    if unique_rows != scores.height:
        raise ValueError("Duplicate identifier rows found.")

    for column in ("T2", "SPE"):
        values = scores[column].to_numpy()

        if not np.all(np.isfinite(values)):
            raise ValueError(
                f"Non-finite values found in {column}."
            )

        if np.any(values < 0):
            raise ValueError(
                f"Negative values found in {column}."
            )

```

)

```
def run() -> pl.DataFrame:
    """Fit the PCA monitor and write results/scores_pca.parquet."""

    train, scored = load()

    standardizer = Standardizer(train)
    standardized_train = standardizer.transform(train)
    standardized_scored = standardizer.transform(scored)

    model = fit_pca(standardized_train)

    explained_ratio = (
        model.all_eigenvalues
        / model.all_eigenvalues.sum()
    )
    cumulative = np.cumsum(explained_ratio)

    print(f"Retained PCA components: {model.n_components}")
    print(
        "Cumulative explained variance: "
        f"{model.cumulative_explained_variance:.6f}"
    )

    if model.n_components > 1:
        print(
            "Variance with k - 1 components: "
            f"{cumulative[model.n_components - 2]:.6f}"
        )

    scores = score(standardized_scored, model)
    validate_scores(scores)

    RESULTS.mkdir(exist_ok=True)
    output_path = RESULTS / "scores_pca.parquet"
    scores.write_parquet(output_path)

    print(f"{scores.height:,} PCA rows scored")
    print(f"Saved PCA scores to {output_path}")

    return scores

if __name__ == "__main__":
    run()
```

Code: plot_contributions.py

```
"""Plot the channels that drive the SPE and ridge alarms for a few faults, as shares of the mean
statistic."""

from __future__ import annotations

import argparse

import matplotlib
import matplotlib.pyplot as plt
import polars as pl

from common import FIGURES
from diagnosis import TOP, mean_table

matplotlib.use("Agg")

# Downs and Vogel (1993); streams in brackets.
```

```

DESCRIPTION = {
    "xmeas_1": "A feed (1)", "xmeas_2": "D feed (2)", "xmeas_3": "E feed (3)", "xmeas_4": "A and C feed (4)"
    ,
    "xmeas_5": "recycle flow (8)", "xmeas_6": "reactor feed rate (6)", "xmeas_7": "reactor pressure",
    "xmeas_8": "reactor level", "xmeas_9": "reactor temperature", "xmeas_10": "purge rate (9)",
    "xmeas_11": "separator temperature", "xmeas_12": "separator level", "xmeas_13": "separator pressure",
    "xmeas_14": "separator underflow (10)", "xmeas_15": "stripper level", "xmeas_16": "stripper pressure",
    "xmeas_17": "stripper underflow (11)", "xmeas_18": "stripper temperature", "xmeas_19": "stripper steam
    flow",
    "xmeas_20": "compressor work", "xmeas_21": "reactor cooling water outlet temp.",
    "xmeas_22": "separator cooling water outlet temp.",
    **{f"xmeas_{23 + i}": f"reactor feed {c} (6)" for i, c in enumerate("ABCDEF")},
    **{f"xmeas_{29 + i}": f"purge gas {c} (9)" for i, c in enumerate("ABCDEFGH")},
    **{f"xmeas_{37 + i}": f"product {c} (11)" for i, c in enumerate("DEFGH")},
    "xmv_1": "D feed flow (2)", "xmv_2": "E feed flow (3)", "xmv_3": "A feed flow (1)",
    "xmv_4": "A and C feed flow (4)", "xmv_5": "compressor recycle valve", "xmv_6": "purge valve (9)",
    "xmv_7": "separator pot liquid flow (10)", "xmv_8": "stripper liquid product flow (11)",
    "xmv_9": "stripper steam valve", "xmv_10": "reactor cooling water flow", "xmv_11": "condenser cooling
    water flow",
}

# Chiang, Russell and Braatz (2000), Table 1.
FAULT_NAME = {
    1: "A/C feed ratio, B composition constant (4), step", 2: "B composition, A/C ratio constant (4), step",
    3: "D feed temperature (2), step", 4: "reactor cooling water inlet temperature, step",
    5: "condenser cooling water inlet temperature, step", 6: "A feed loss (1), step",
    7: "C header pressure loss (4), step", 8: "A, B, C feed composition (4), random",
    9: "D feed temperature (2), random", 10: "C feed temperature (4), random",
    11: "reactor cooling water inlet temperature, random", 12: "condenser cooling water inlet temperature,
    random",
    13: "reaction kinetics, slow drift", 14: "reactor cooling water valve, sticking",
    15: "condenser cooling water valve, sticking",
}

BAR = "#2a78d6"
INK, MUTED, GRID = "#0b0b0b", "#52514e", "#e1e0d9"

def shares(means: pl.DataFrame) -> pl.DataFrame:
    """Add each channel's share of the mean statistic, the sum of the mean contributions of its fault and
    detector."""
    return means.with_columns(share=pl.col("contribution") / pl.col("contribution").sum().over("fault", "det
    ector"))

def panel(ax: plt.Axes, g: pl.DataFrame, detector: str, right: float) -> None:
    """Draw the top channels of one fault and detector as horizontal bars, largest on top."""
    ax.set_xlim(0, right)
    ax.xaxis.set_major_locator(matplotlib.ticker.MultipleLocator(0.25))
    ax.xaxis.set_major_formatter(matplotlib.ticker.PercentFormatter(1.0, decimals=0))
    ax.tick_params(axis="x", labelsize=6.5, colors=MUTED, length=0)
    ax.spines[["top", "right", "left"]].set_visible(False)
    ax.spines["bottom"].set_color(GRID)
    ax.grid(axis="x", color=GRID, lw=0.6)
    ax.set_axisbelow(True)
    if g.is_empty():
        ax.set_yticks([])
        ax.set_title(f"{detector}: no alarm sample after sample 20", fontsize=7, color=MUTED, loc="right", p
        ad=3)
        return
    g = g.sort("share", descending=True).head(TOP)
    y = list(range(g.height))[:-1]
    ax.barh(y, g["share"].to_numpy(), height=0.62, color=BAR)
    for yi, share in zip(y, g["share"]):
        ax.text(share + 0.012, yi, f"{share:.0%}", va="center", fontsize=6.5, color=INK)
    ax.set_yticks(y, [f"{c} {DESCRIPTION[c]}" for c in g["channel"]], fontsize=6.5, color=INK)
    ax.tick_params(axis="y", length=0, pad=3)

```

```
ax.set_title(f"{detector} • {g['samples'][0]:,} alarm samples", fontsize=7, color=MUTED, loc="right", pad=3)
```

```
def run(faults: list[int]) -> None:
    """Write figures/contributions.png: the top channels of each fault, SPE on the left and ridge on the right."""
    means = shares(mean_table()).filter(pl.col("fault").is_in(faults))
    top = means.sort("share", descending=True).group_by("fault", "detector").head(TOP)
    right = min(1.0, (top["share"].max() or 0.0) + 0.12)

    height = 0.6 + 1.55 * len(faults)
    fig, axes = plt.subplots(len(faults), 2, figsize=(6.5, height), sharex=True, squeeze=False, layout="constrained")
    fig.get_layout_engine().set(hspace=0.2, rect=(0, 0, 1, 1 - 0.22 / height))
    for row, fault in zip(axes, faults):
        for ax, detector in zip(row, ("SPE", "ridge")):
            panel(ax, top.filter((pl.col("fault") == fault) & (pl.col("detector") == detector)), detector, right)
            title = row[0].annotate(f"IDV({fault}): {FAULT_NAME.get(fault, 'unknown')}", xy=(0.0, 1.0),
                                   xycoords=("figure fraction", "axes fraction"), xytext=(2, 16),
                                   textcoords="offset points", fontsize=8, color=INK, weight="bold")
            title.set_in_layout(False)
    fig.supxlabel("share of the mean statistic over the alarm samples after sample 20", fontsize=6.5, color=MUTED)

    FIGURES.mkdir(exist_ok=True)
    fig.savefig(FIGURES / "contributions.png", dpi=200)

if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--faults", type=int, nargs="+", default=[4, 6, 10])
    args = parser.parse_args()
    run(args.faults)
```

Code: plot_detectors.py

```
"""Plot T2, SPE and the ridge score of one faulty run, each against its threshold in results/thresholds.csv."""

from __future__ import annotations

import argparse

import matplotlib
import matplotlib.pyplot as plt
import numpy as np
import polars as pl

from common import FAULT, FAULT_START, FIGURES, KEYS, MINUTES_PER_SAMPLE, RESULTS, RUN, SAMPLE

matplotlib.use("Agg")

PANELS = [("T2", "T2", "$T^2$"), ("SPE", "SPE", "SPE"), ("ridge", "score", "ridge score")]

def run(fault: int, sim_run: int) -> None:
    """Write figures/detectors_fault{fault}_run{sim_run}.png."""
    thresholds = dict(pl.read_csv(RESULTS / "thresholds.csv").iter_rows())
    pca = pl.read_parquet(RESULTS / "scores_pca.parquet")
    ridge = pl.read_parquet(RESULTS / "scores_ridge.parquet")
    g = (pca.join(ridge, on=KEYS, how="left")
         .filter((pl.col(FAULT) == fault) & (pl.col(RUN) == sim_run)).sort(SAMPLE))
    hours = g[SAMPLE].to_numpy() * MINUTES_PER_SAMPLE / 60
    onset = FAULT_START * MINUTES_PER_SAMPLE / 60
```

```

fig, axes = plt.subplots(3, 1, figsize=(6.5, 3.6), sharex=True)
for ax, (detector, column, label) in zip(axes, PANELS):
    values = g[column].to_numpy()
    ax.plot(hours, values, color="#2a6fb0", lw=1.0)
    ax.set_ylim(0, 1.3 * max(values[~np.isnan(values)].max(), thresholds[detector]))
    ax.axhline(thresholds[detector], color="#555555", ls="--", lw=1,
               label=f"threshold {thresholds[detector]:.1f}")
    ax.axvline(onset, color="#999999", ls=":", lw=1)
    ax.set_ylabel(label, fontsize=9)
    ax.tick_params(labelsize=8)
    ax.spines[["top", "right"]].set_visible(False)
    ax.grid(axis="y", color="#e5e5e5", lw=0.6)
    ax.legend(loc="upper right", fontsize=7.5, frameon=False)
axes[0].text(onset, axes[0].get_ylim()[1], " fault on", ha="left", va="top", fontsize=8, color="#333333")
)
axes[0].set_title(f"IDV({fault}), run {sim_run}", fontsize=10)
axes[-1].set_xlabel("time (h)")
fig.tight_layout()

FIGURES.mkdir(exist_ok=True)
fig.savefig(FIGURES / f"detectors_fault{fault}_run{sim_run}.png", dpi=200)

```

```

if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--fault", type=int, default=4)
    parser.add_argument("--run", type=int, default=1)
    args = parser.parse_args()
    run(args.fault, args.run)

```

Code: plot_ridge.py

```

"""Plot the ridge score of one faulty run against the 99th-percentile validation threshold."""

from __future__ import annotations

import argparse

import matplotlib
import matplotlib.pyplot as plt
import numpy as np
import polars as pl

from common import FAULT, FAULT_START, FIGURES, MINUTES_PER_SAMPLE, RESULTS, RUN, SAMPLE, VALIDATION

matplotlib.use("Agg")

def threshold(scores: pl.DataFrame) -> float:
    """Return numpy.quantile(score, 0.99) over the validation runs."""
    val = scores.filter((pl.col(FAULT) == 0) & pl.col(RUN).is_between(*VALIDATION))
    return float(np.quantile(val["score"].to_numpy(), 0.99))

def run(fault: int, sim_run: int) -> None:
    """Write figures/ridge_fault{fault}_run{sim_run}.png."""
    scores = pl.read_parquet(RESULTS / "scores_ridge.parquet")
    limit = threshold(scores)
    g = scores.filter((pl.col(FAULT) == fault) & (pl.col(RUN) == sim_run)).sort(SAMPLE)
    hours = g[SAMPLE].to_numpy() * MINUTES_PER_SAMPLE / 60

    fig, ax = plt.subplots(figsize=(6.5, 3.0))
    ax.plot(hours, g["score"].to_numpy(), color="#2a6fb0", lw=1.2, label="score")
    ax.axhline(limit, color="#555555", ls="--", lw=1, label=f"threshold {limit:.1f}")
    ax.axvline(FAULT_START * MINUTES_PER_SAMPLE / 60, color="#999999", ls=":", lw=1)
    ax.text(FAULT_START * MINUTES_PER_SAMPLE / 60, ax.get_ylim()[1], " fault on", ha="left", va="top",
           fontsize=8, color="#333333")

```

```

ax.set_xlabel("time (h)")
ax.set_ylabel("ridge score")
ax.set_title(f"Ridge forecast-residual score, IDV({fault}), run {sim_run}", fontsize=10)
ax.spines[["top", "right"]].set_visible(False)
ax.grid(axis="y", color="#e5e5e5", lw=0.6)
ax.legend(loc="upper right", fontsize=8, frameon=False)
fig.tight_layout()

FIGURES.mkdir(exist_ok=True)
fig.savefig(FIGURES / f"ridge_fault{fault}_run{sim_run}.png", dpi=200)

```

```

if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--fault", type=int, default=4)
    parser.add_argument("--run", type=int, default=1)
    args = parser.parse_args()
    run(args.fault, args.run)

```

Summary line

```

{"team": "\nminiproject20", "andrew_ids": ["wenwenw", "junwooh", "bolanz", "pbu"], "generated": "2026-10-09
20:13 EDT", "auto_score": 10.0, "auto_of": 10, "held_for_ta": 0.0, "total": 15, "script_sha256":
"744445fa94498271372f89d7a8053499fd20999c7754842f770eb7bd5f4a157a"}

```